# CSE 256 Fall 2026, UCSD: PA1

Yung-Hsin Liu — PID: 200478775

October 23, 2026

## 1 Part 1a: Deep Averaging Network

### 1.1 Implementation

Each sentence is converted to a list of word IDs by looking up each word in the GloVe vocabulary. Words that are not in GloVe are mapped to ID 1 (the UNK token). To make all sentences in a dataset the same length, we find the longest list of IDs and pad every other list with 0 (the PAD token) up to that length. The embedding layer is initialized with the GloVe vectors and fine tuned during training. For each sentence, we average the embeddings of its words, ignoring PAD positions, so the average is the sum of the real word vectors divided by the number of real words. The averaged vector then goes through dropout $\rightarrow$ Linear$(d, 100)$ $\rightarrow$ ReLU $\rightarrow$ dropout $\rightarrow$ Linear$(100, 2)$ $\rightarrow$ LogSoftmax, where $d$ is the embedding size (50 or 300). We train with NLLLoss and Adam (learning rate 0.001), batch size 16, for 20 epochs.

### 1.2 Results

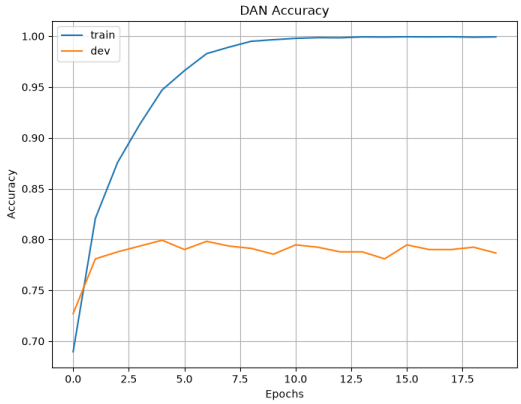
| Embeddings | Setting | Final train | Final dev | Best dev | Time |
|---|---|---|---|---|---|
| GloVe 50d | dropout 0.0 | 99.9% | 78.7% | 79.9% | 11.9s |
| GloVe 50d | dropout 0.3 (run 1) | 97.6% | **79.1%** | 80.3% | 11.5s |
| GloVe 50d | dropout 0.3 (run 2) | 97.5% | 79.0% | 80.2% | 13s |
| GloVe 300d | dropout 0.3 | 99.4% | 78.8% | 82.8% | 63.5s |
| GloVe 300d | dropout 0.3, frozen | 77.9% | 78.3% | 79.4% | 5.4s |
| GloVe 300d | dropout 0.3, hidden 300 | 99.6% | 78.3% | 81.1% | 60.0s |

Table 1: DAN results with GloVe initialization. Unless noted: hidden size 100, Adam lr 0.001, 20 epochs.

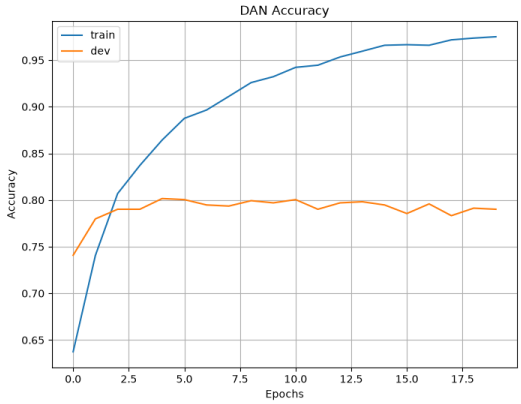
### 1.3 Discussion

All GloVe settings reach the 77% target. The best final dev accuracy is 79.1% (50d, dropout 0.3). The highest dev accuracy at any epoch is 82.8% (300d, dropout 0.3), run that run drops back to 78.8% by epoch 20.

Without dropout the model will overfit. Train accuracy reaches 99.9% while dev accuracy stays at 78.7%. Dropout forces the model to rely on broader patterns instead of specific combinations. With dropout 0.3, train accuracy drops by 2.3 points and dev accuracy rises by 0.4 points.




(a) No dropout




(b) Dropout 0.3

Figure 1: Train and dev accuracy per epoch, GloVe 50d.

The 300d embeddings do not improve the final dev accuracy. They take about 5 times longer to train and overfit more. With the same training sentences, 300d has more learnable numbers in the embedding layer so it has more room to memorize. Freezing the embeddings removes the overfitting gap, which suggests most of the memorization happens in the embedding layer. However, it also lowers the ceiling since the vectors cannot adapt to sentiment. We also found that a larger hidden layer does not help.

Two runs of the same setting differ by only 0.1% in final dev accuracy, but most settings differ from each other by less than 1%, so the difference is not that significant. We consider the dev accuracy of the final epoch as our main number as the best epoch is picked using the dev set and is slightly more optimistic.

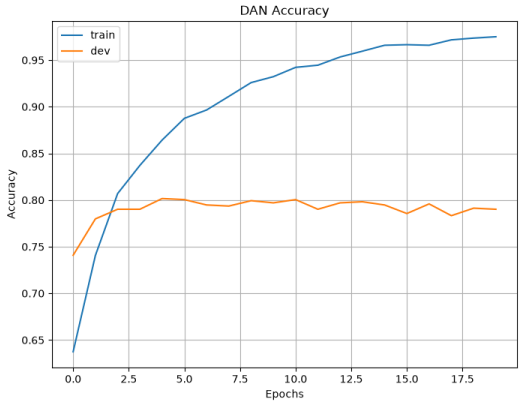
## 2 Part 1b: Randomly Initialized Embeddings

| Embeddings | Dropout | Final train | Final dev | Best dev | Time |
|---|---|---|---|---|---|
| GloVe 50d | 0.3 | 97.6% | **79.1%** | 80.3% | 11.5s |
| Random 50d | 0.0 | 100.0% | 75.3% | 76.6% | 17s |
| Random 50d | 0.3 (run 1) | 94.8% | 78.0% | 78.0% | 22s |
| Random 50d | 0.3 (run 2) | 94.3% | 78.4% | 78.4% | 12s |
| Random 300d | 0.0 | 100.0% | 77.2% | 77.5% | 124s |
| Random 300d | 0.3 | 98.4% | 78.2% | 79.0% | 143s |

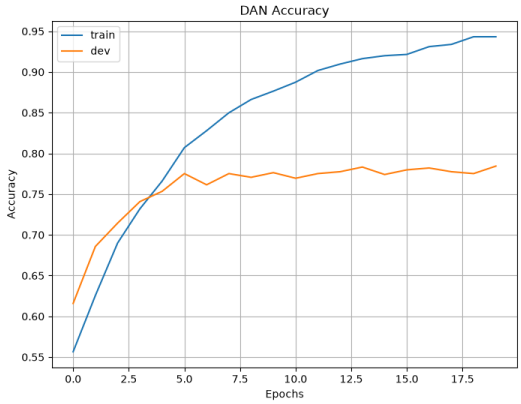
Table 2: GloVe vs. random initialization.

GloVe initialization outperforms random initialization by about 1–3 points in dev accuracy. 79.1% vs. 78.0–78.4% final dev with dropout 0.3, and 78.7% vs. 75.3% without dropout. GloVe was pretrained on a very large text corpus, so its vectors already capture word meaning, while randomly initialized embeddings must be learned from only 6,920 training sentences, where many words only appear a few times.

GloVe also converges much faster (Figure 2). Its dev accuracy is about 74% after the first epoch




(a) GloVe




(b) Random

Figure 2: GloVe vs. Random, same settings (50d, dropout=0.3)

and 79% after the third, while the random model starts at about 62% and is still around 72% after three epochs. With random vectors, the model has to learn what each word means and how to classify sentiment at the same time.

Dropout helps the random model more than the GloVe model. It raises random 50d from 75.3% to 78.0–78.4% (about 3 points), compared to 0.4 points for GloVe. Thi might be because the random model overfits more (100% train accuracy without dropout). Random 300d embeddings reach a similar dev accuracy but take 7–12 times longer to train.

# 3 Part 2: Byte Pair Encoding

## 3.1 Implementation

BPE is learned from `train.txt` only. Each word first split into its characters plus an end of word symbol `</w>` at the end. In each step, we count every pair of neighboring symbols across the training words. Each pair is counted as many times as its word appears, and we merge the most frequent pair everywhere it occurs. Each merge is recorded in an ordered list, and learning stops after the number of merges we set. To split a word, we start from its characters and repeatedly apply the merge that was learned the earliest that matches, since merges that are learned later are built from the pieces that earlier merges create.

The subword vocabulary contains every piece that appears in the tokenized training data, plus PAD (ID 0) and UNK (ID 1). We use the same DAN as in Part 1 with dropout = 0.3, but with randomly initialized 50d embeddings, since GloVe has no vectors for subword pieces. Because BPE is greedy (each merge depends only on the data and the earlier merges), the first $N$ merges are the same no matter how many are learned in total. We therefore learn the merges once, save them, and reuse the first $N$ for each vocabulary size.

## 3.2 Results

See Table 3 and Figure 3

| Merges | Vocab | Max pieces | Final train | Final dev | Best dev | Time |
|---|---|---|---|---|---|---|
| 500 | 563 | 115 | 76.8% | 71.8% | 72.8% | 8s |
| 1000 | 1,055 | 101 | 80.2% | 75.3% | 75.8% | 9s |
| 2000 (run 1) | 2,043 | 89 | 85.4% | 75.0% | 75.0% | 9s |
| 2000 (run 2) | 2,043 | 89 | 85.8% | 75.8% | 76.5% | 7s |
| 5000 | 4,886 | 77 | 91.8% | 75.9% | **76.6%** | 10s |
| 10000 | 9,032 | 64 | 95.4% | 74.9% | 75.5% | 13s |
| Word-level (random) | ∼14,900 | 52 | 94.8% | 78.0% | 78.0% | 22s |

Table 3: Subword DAN for different numbers of BPE merges (random 50d embeddings, dropout 0.3).

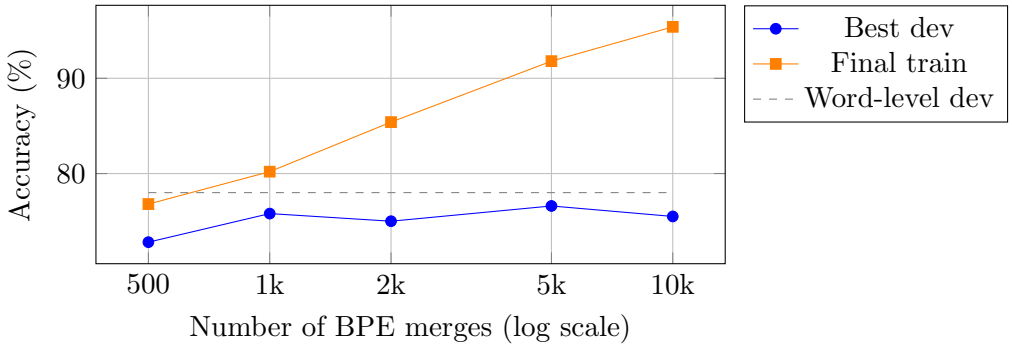



Figure 3: Effect of vocabulary size (run 1 values).

### 3.3 Discussion

With only 500 merges the model underfits, as you can see, even train accuracy is only 76.8%. The pieces are small and shared by many unrelated words, so averaging them makes the meaning of the sentence more unclear. From 1,000 to 10,000 merges, dev accuracy stays roughly the same while train accuracy keeps increasing, so larger vocabularies mainly add overfitting (Figure 3).

The vocabulary is smaller than the number of merges plus characters because some merged pieces are only intermediate steps. For example, `th` is created early but later absorbed into `the</w>`, so it never appears in the final tokenized training data. The maximum number of pieces per sentence cannot go below 52, the length of the longest training sentence in words, because BPE never merges across word boundaries, so every word becomes at least one piece.

No subword setting beats the word level random DAN (78.0%). Subwords help with unseen words, and 5.7% of dev words never appear in training (59% of dev sentences contain at least one). However, in a DAN an unseen word is only one of about 20 vectors being averaged, so handling it better changes little. At the same time, splitting every word into pieces shared across unrelated words makes each word's representation coarser, and averaging loses which pieces belong together.

In terms of efficiency, fewer merges give a smaller vocabulary and embedding table (563 vs. 9,032 entries) and faster training (8s vs. 13s), but longer inputs (115 vs. 64 maximum pieces per sentence).

# 4 Part 3: Understanding Skip-Gram

## Q1

Sentences: **the dog**, **the cat**, **a dog**. Window $k = 1$.

Training pairs $(x, y)$: (the, dog), (dog, the), (the, cat), (cat, the), (a, dog), (dog, a).

**3a.** Two pairs have $x =$ the: (the, dog) and (the, cat). The maximum likelihood estimate of the probabilities:

$$P(\text{dog} \mid \text{the}) = \tfrac{1}{2}, \quad P(\text{cat} \mid \text{the}) = \tfrac{1}{2}, \quad P(\text{a} \mid \text{the}) = 0, \quad P(\text{the} \mid \text{the}) = 0.$$

**3b.** With $\mathbf{c}_{\text{dog}} = \mathbf{c}_{\text{cat}} = (0, 1)$ and $\mathbf{c}_{\text{a}} = \mathbf{c}_{\text{the}} = (1, 0)$, choose $\mathbf{v}_{\text{the}} = (0, 5)$. The scores are $\mathbf{v}_{\text{the}} \cdot \mathbf{c}_{\text{dog}} = \mathbf{v}_{\text{the}} \cdot \mathbf{c}_{\text{cat}} = 5$ and $\mathbf{v}_{\text{the}} \cdot \mathbf{c}_{\text{a}} = \mathbf{v}_{\text{the}} \cdot \mathbf{c}_{\text{the}} = 0$, so

$$P(\text{dog} \mid \text{the}) = P(\text{cat} \mid \text{the}) = \frac{e^5}{2e^5 + 2} \approx \frac{148}{298} \approx 0.497, \qquad P(\text{a} \mid \text{the}) = P(\text{the} \mid \text{the}) = \frac{1}{2e^5 + 2} \approx 0.003,$$

which is all within 0.01 of the optimum.

This is optimal because **dog** and **cat** have the same context vector, so for any choice of $\mathbf{v}_{\text{the}}$ they will always have the same probability. As for **a** and **the**, they never appear as neighbors of **the** in the training data, so we try to push their probability toward 0. We do so by setting $\mathbf{v}_{\text{the}} = (0, 5)$. This gives**dog** and **cat** a score of 5 and **a** and **the** a score of 0. Since $e^5 \approx 148$ is much bigger than $e^0 = 1$, almost all the probability goes to **dog** and **cat**. Because $e^x$ is always positive, **a** and **the** can never get exactly 0. We can only get closer by making the 5 larger, and 5 is already enough to be within 0.01.

## Q2

Sentences: the dog, **the cat**, **a dog**, **a cat**.

**3c.** (the, dog), (dog, the), (the, cat), (cat, the), (a, dog), (dog, a), (a, cat), (cat, a).

**3d.** The optimal probabilities put $\frac{1}{2}$ on each word of the other group and 0 on the same group: $P(\text{dog} \mid x) = P(\text{cat} \mid x) = \frac{1}{2}$ for $x \in \{\text{the}, \text{a}\}$, and $P(\text{the} \mid x) = P(\text{a} \mid x) = \frac{1}{2}$ for $x \in \{\text{dog}, \text{cat}\}$.

| Word | $\mathbf{v}$ (center) | $\mathbf{c}$ (context) |
|---|---|---|
| the | $(0, 5)$ | $(1, 0)$ |
| a | $(0, 5)$ | $(1, 0)$ |
| dog | $(5, 0)$ | $(0, 1)$ |
| cat | $(5, 0)$ | $(0, 1)$ |

Table 4: Near-optimal word and context vectors.

Check for center word dog: scores are 5 for the and a, and 0 for dog and cat, so $P(\text{the} \mid \text{dog}) = P(\text{a} \mid \text{dog}) \approx 0.497$ and $P(\text{dog} \mid \text{dog}) = P(\text{cat} \mid \text{dog}) \approx 0.003$. By symmetry the same holds for cat, and for the and a (as in 3b).

The four words split into two groups, $\{\textbf{dog}, \textbf{cat}\}$ and $\{\textbf{a}, \textbf{the}\}$. Every word's neighbors are always in the other group. We give each group its own dimension: the context vectors of **a** and **the**

use the first dimension, $(1, 0)$, and those of **dog** and **cat** use the second dimension, $(0, 1)$. Each center vector puts a large value, 5, in the dimension of the other group, so its dot product is 5 with the words that can be its neighbors and 0 with the words in its own group. So each center word gives about 0.5 to each of its two real neighbors and about 0 to the others, which is within 0.01 of the optimum.